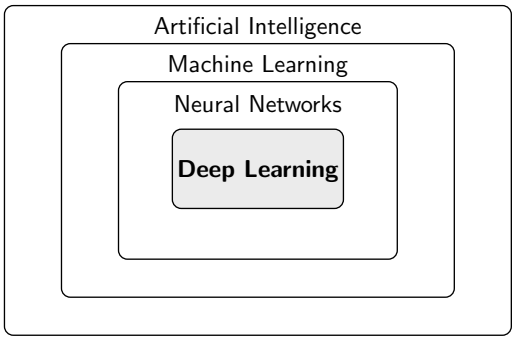
Introduction to Deep Learning

How a network turns inputs into a prediction and learns from its mistakes

DS331: Introduction to Data Science, Data Analysis & Management
Prepared By: Syed Shams Islam

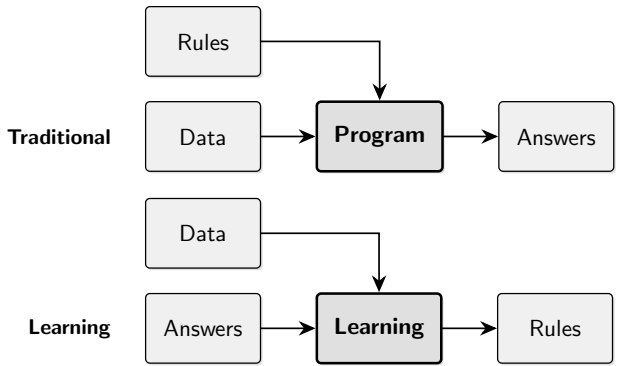
What Is Deep Learning?

- Machines that learn patterns *straight from data* — images, text, sound.
- Built from many simple units (*neurons*) stacked in layers.
- "Deep" just means *many layers*.



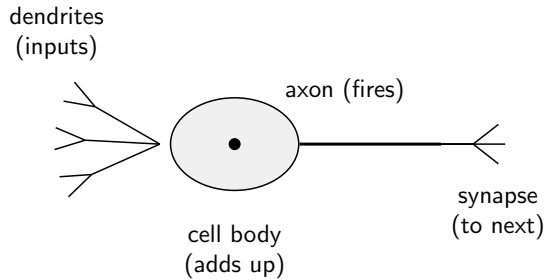
Rules vs. Learning From Examples

- Traditional: humans write the rules.
- Machine learning: learns the rules from examples.



Inspired by the Brain

- Your brain is billions of *neurons* wired together.
- A neuron collects signals, adds them up, and *fires* if they are strong enough.
- The artificial neuron copies this one simple idea.



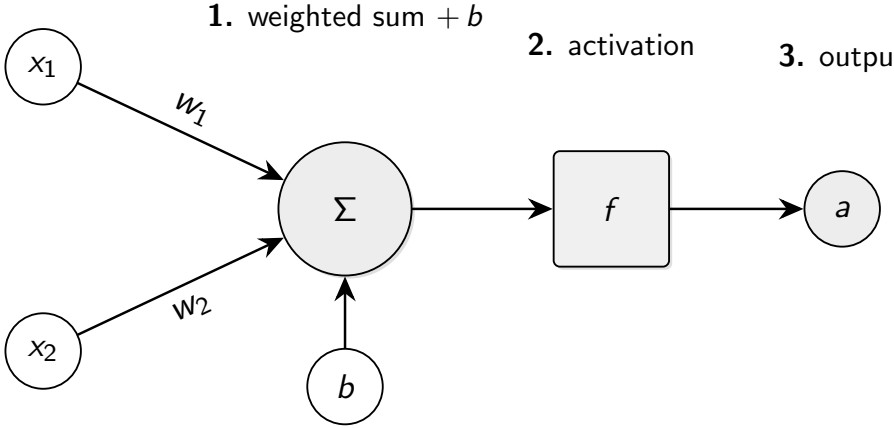
From Neuron to Math

The artificial neuron mirrors the biological one:

Biological neuron	$\longrightarrow$	**Artificial neuron**
Dendrites (incoming signals)	$\longrightarrow$	Inputs $x_1, x_2, \ldots, x_n$
Synapse strength	$\longrightarrow$	Weights $w_1, w_2, \ldots, w_n$
Cell body adds the signals	$\longrightarrow$	Weighted sum + bias
Fires if strong enough	$\longrightarrow$	Activation function
Axon output	$\longrightarrow$	Output a

Inside a Single Neuron

A neuron turns its inputs into one number, in three steps:



One Neuron, Worked by Hand

Will this student pass the exam?

Given (scaled) inputs:

- hours studied $x_1 = 2$, $w_1 = +1.5$
- classes missed $x_2 = 1$, $w_2 = -1.0$
- bias $b = +0.5$

Activation: sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

$$\begin{aligned} z &= (1.5)(2) + (-1.0)(1) + 0.5 \\ &= 2.5 \end{aligned}$$

$$a = \sigma(2.5) = \frac{1}{1 + e^{-2.5}} \approx 0.92$$

$a \approx 0.92$

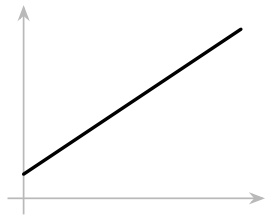
about a 92% chance of passing

What the Weights and Bias Mean

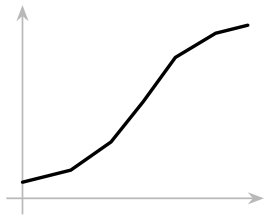
- **Weight** = how much an input matters: large = strong pull, negative = opposes, near zero = ignored.
- **Bias** = how easily the neuron switches on.
- **Learning** is nothing more than finding good weights and biases.

Why We Need an Activation Function

- Without it, stacking neurons only ever makes a *straight line*.
- The activation adds a *bend*, so the network can fit real, curvy patterns.

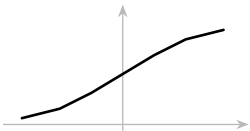


lines only $\Rightarrow$ still a line



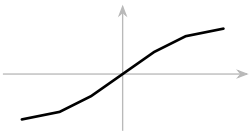
with activation $\Rightarrow$ curves

Common Activation Functions



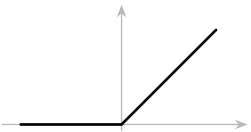
sigmoid

output in $(0, 1)$



tanh

output in $(-1, 1)$



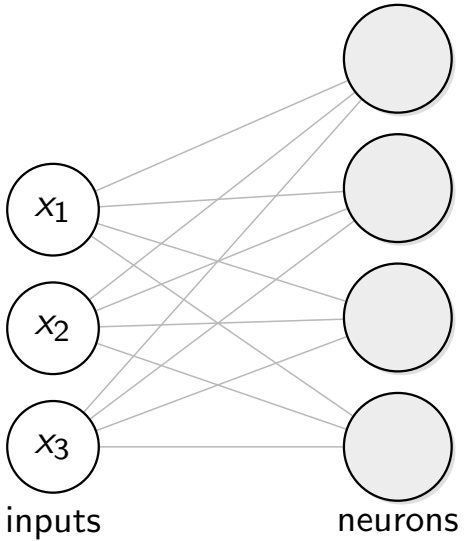
ReLU

$\max(0, z)$ — the default

Sigmoid for probabilities; ReLU is the usual choice inside deep networks.

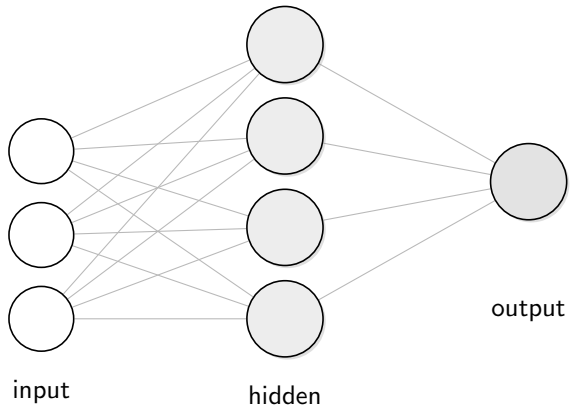
From One Neuron to a Layer

- Line up several neurons on the *same* inputs.
- Each has its own weights, so each learns a *different* pattern.
- Every input to every neuron = a **fully connected** layer.



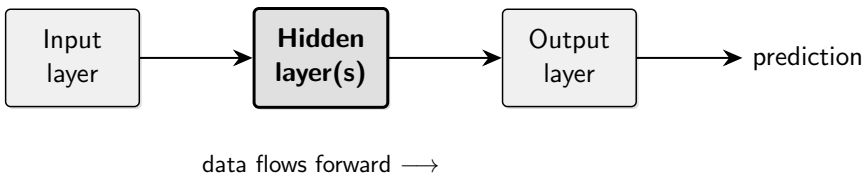
Stacking Layers — The Multilayer Perceptron

- **Input** layer: the features.
- **Hidden** layer(s): build up the patterns.
- **Output** layer: the prediction.
- More hidden layers = "deeper".



The Forward Pass

– Feed the features in; each neuron runs its little formula.
– Outputs flow *forward*, layer by layer, to the prediction. One pass = one guess.



The Loss Function

- A fresh network guesses randomly — we need one number for *how wrong* it is: the **loss**.
- Predicting numbers: **Mean Squared Error**, the average of $(\text{actual} - \text{predicted})^2$.
- Yes/no answers: **Cross-Entropy**. Smaller loss = better.

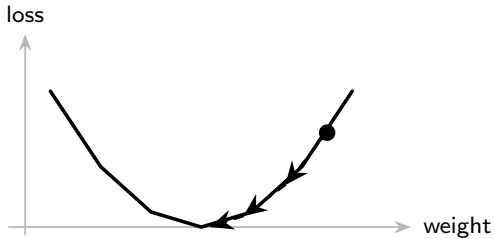
A tiny MSE:

actual	predicted	error^2
3	2	1
5	7	4

$$\text{MSE} = \frac{1+4}{2} = 2.5$$

Gradient Descent — Walking Downhill

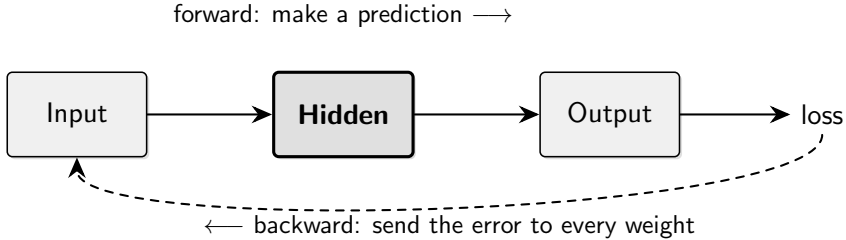
- Training = tune the weights to make the loss small.
- Picture the loss as a *valley*; the slope (**gradient**) points uphill.
- Step the other way — downhill — again and again.
- **Learning rate** = step size.



each step = learning rate

Backpropagation — Sending the Blame Back

- To step downhill we need the slope for *every* weight.
- Make a prediction, measure the error, then send it *backward* through the layers.
- Each weight learns how much it caused the mistake — that is its slope.



The Optimizer — Updating the Weights

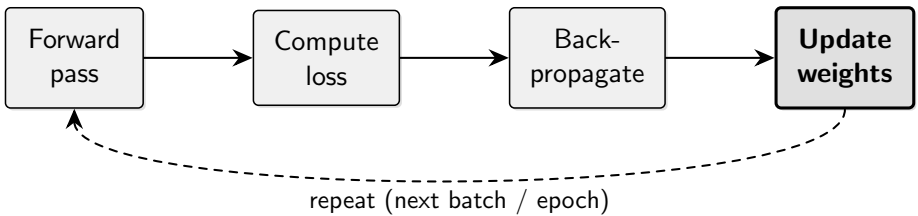
- The optimizer takes the step for each weight:

$$w \leftarrow w - \eta \cdot (\text{slope}), \qquad \eta = \text{learning rate}.$$

- Example: $w = 0.8$, slope $= +2$, $\eta = 0.1 \Rightarrow w \leftarrow 0.8 - 0.2 = 0.6$.
- **SGD** is the basic version; **Adam** adapts the step and is the common default.

The Training Loop

- Repeat: forward pass $\rightarrow$ loss $\rightarrow$ backpropagate $\rightarrow$ update.
- An **epoch** is one full pass over the data; do many, and the loss falls.



Overfitting — Memorizing vs. Learning

- A big network can *memorize* the training data yet fail on new data.
- Catch it with a held-out **validation set**: watch the training–validation gap.
- Fixes: more data, a smaller network, early stopping, or *dropout*.

What You Set vs. What It Learns

You choose
(hyperparameters)

- Number of layers
- Neurons per layer
- Activation function
- Learning rate, epochs

It learns
(parameters)

- The weights
- The biases
- (often millions of them)

Where Deep Learning Shines

- **Images:** convolutional networks — edges $\rightarrow$ shapes $\rightarrow$ objects.
- **Text:** Transformers — the engine behind large language models (our LLM class).
- **Also:** speech, recommendations, medicine, games.
- The core idea never changes: neurons, a forward pass, a loss, backprop, an optimizer.

Recap — The Whole Picture

- A neuron weighs its inputs, adds a bias, applies an activation:

$$z = w_1 x_1 + \cdots + b, \qquad a = f(z).$$

- Stack neurons into layers $\rightarrow$ a network.
- **Forward pass** $\rightarrow$ prediction $\rightarrow$ **loss** (how wrong).
- **Backprop** $\rightarrow$ slopes $\rightarrow$ the **optimizer** nudges every weight downhill.
- Repeat until the loss is small — that is learning.